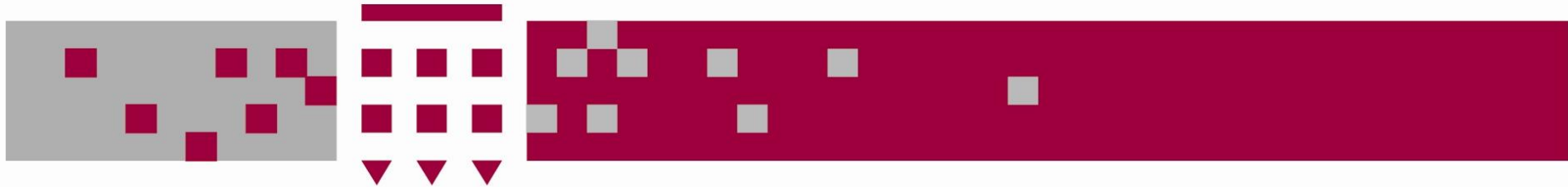


UNIVERSITY OF WESTMINSTER



## **5COSC019W – Object Oriented Programming Week 09**

Dr. Barbara Villarini

[b.villarini@westminster.ac.uk](mailto:b.villarini@westminster.ac.uk)

# Summary

- Introduction to Error and error handling
- Exception
- `try` and `catch` statements
- Multiple `catch` statements
- Using `finally`
- Declaring and Throwing Exceptions
- Creating custom Exceptions

# Introduction

- It is extremely rare to write perfect code the first time even though user have high expectation for the code we produce.
- **Keep in mind:** Users will use our programs in unexpected ways.
- Due to design errors or coding errors, our programs may fail in unexpected ways during execution
- It is our responsibility to produce quality code that does not fail unexpectedly.



# Errors and Error Handling

- Some typical causes of errors:
  - Memory errors (i.e. memory incorrectly allocated, memory leaks, “null pointer”) → C++
  - File system errors (i.e. disk is full, disk has been removed)
  - Network errors (i.e. network is down, URL does not exist)
  - Calculation errors (i.e. divide by 0)
  - Array errors (i.e. accessing element -1)
  - Conversion errors (i.e. convert ‘q’ to a number)
  - Can you think of some others?



## Possible strategies

- Ignore the problem – **not good Idea!!!**
- Check for potential problems and abort the program when you find the problem – **no good solution!**



➡ Check for potential problems, catch the mistake, and attempt to fix the problem.

- Throw an exception. – **This is the preferred way to handle the situation**

# Exceptions

- Most of the OO languages provides features called **exceptions**.
- Exception is an indication of problem during execution.
- Uses of exception handling
  - Process exceptions from program components
  - Handle exceptions in a uniform manner in large projects
  - Remove error-handling code from “main line” of execution
- In Java, C++, C#, Objective-C and Visual Basic, etc. exceptions are handled by the keywords **try**, **catch** and **throw**.

# Handling Exception with Try/Catch block

- Exception handling mechanism:
  - Find the problem (**try** block : Hit the exception)
  - Inform that an error has occurred (**Throw** the exception)
  - Receive the error information (**Catch** the exception)
  - Take corrective action (handle the exception)



# Exception handling mechanism

It is basically built upon three keywords

- Try
- Throw
- Catch

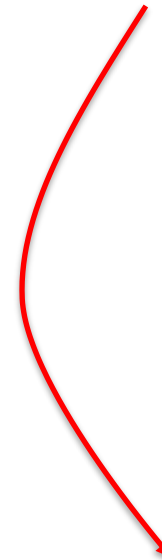
Exception object

## **Try Block**

Detect and throws  
an exception

## **Catch Block**

Catch and handle  
The exception





# Using try and catch

- A structure for a Java try/catch block is the follow:

```
try {  
  
    // possible nasty code  
  
} catch (Exception e){  
  
    // code to handle the exception  
  
}
```

- A `try` followed by any number of `catch` blocks



## Example

```
try {  
    // possible nasty code  
    count = 0;  
    count = 5/count;  
  
} catch (ArithmeticException e) {  
  
    // code to handle the exception  
    System.out.println(e.getMessage());  
    count = 1;  
}  
System.out.println("The Exception is handled");
```



# Using multiple catch statements

- You can associate more than one `catch` statement with a `try`.
- Each `catch` must catch a different type of exception.

```
// Use multiple catch statements
class ExcDemo4 {
    public static void main(String args[]) {
        // Here, number is longer than denominator
        int numer[] = {4, 8, 16, 32, 64, 128, 256, 512};
        int denom[] = {2, 0, 4, 4, 0, 8};

        for (int i = 0; i < numer.length; i++){
            try{
                System.out.println(numer[i] + " / " +
                                    denom[i] + " is " +
                                    numer[i] / denom[i]);
            }
            catch (ArithmeticException exc){
                // Catch the exception
                System.out.println ("Can't divide by zero!");
            }
            catch (ArrayIndexOutOfBoundsException exc){
                // Catch the exception
                System.out.println ("No matching element found.");
            }
        }
    }
}
```



# Output

```
4 / 2 is 2
```

```
Can't divide by zero!
```

```
16 / 4 is 4
```

```
32 / 4 is 8
```

```
Can't divide by zero!
```

```
128 / 8 is 16
```

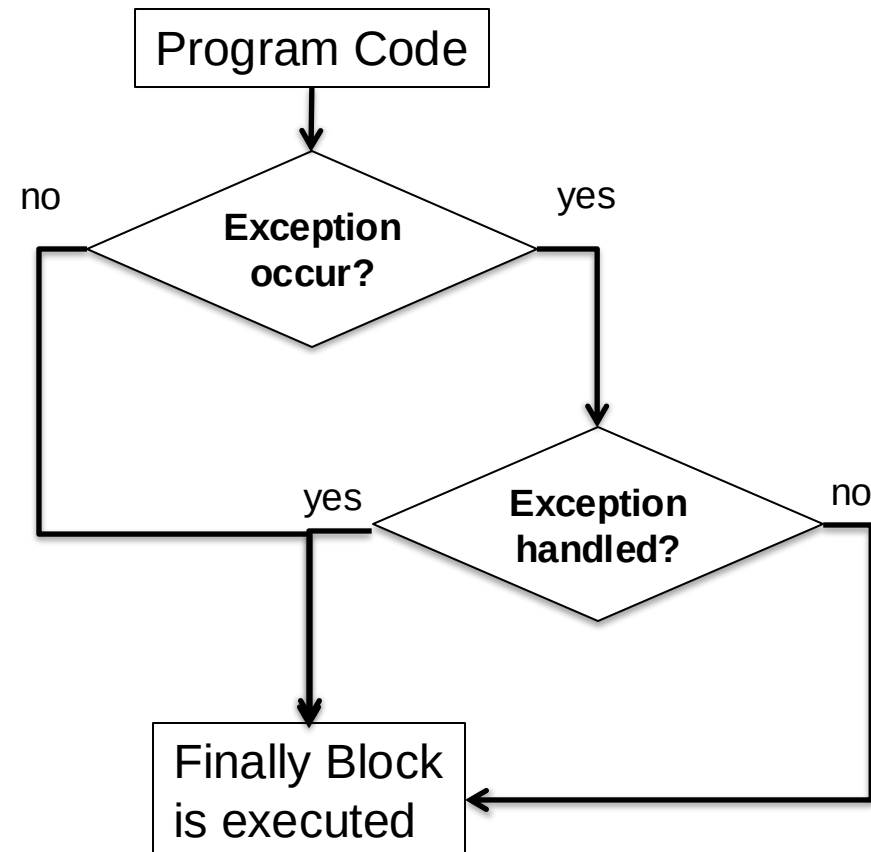
```
No matching element found.
```

```
No matching element found.
```

# Using Finally in Java

- A finally block will be always be executed regardless of what else happens.

```
try {  
    f() ;  
}  
catch (MyException e) // optional {  
    // Do something  
}  
finally {  
    // Guaranteed to execute  
    // this whatever happens.  
}
```



# Throwable class in Java

- When an exception takes place, the JVM creates an exception object to identify the type of exception that occurred
- The **Throwable** class is the super class of all error and exception types generated by the JVM or java programs
- Three **Throwable** subclass categories are possible:
  - **Error** (extends Throwable): Serious error that is not usually recoverable.
  - **Exceptions** (extends Throwable): Error that must be caught and recovered from.
  - **Runtime Exceptions** (extends Exception) : Error that may be caught if desired.

# Checked and Unchecked Exception

- Unchecked Exceptions:
  - They are subclasses of Runtime Exception.
  - *The compiler does not check for these exceptions*, and so a method is not obliged to establish a policy for the unchecked exceptions thrown by its implementation
- Checked Exceptions:
  - They are subclasses of Exception.
  - A checked exception must be caught somewhere in the code, otherwise, it will not compile. So, a method is obliged to establish a policy for all checked exceptions thrown by its implementation.
  - They represent conditions that are reasonably expected to occur, typically representing invalid conditions in areas outside the immediate control of the program (invalid user input, etc).



## Catching Subclass Exception

- A `catch` clause for a superclass will also match any of its subclass.
- Since Superclass of all exceptions is **Throwable**, if you want to catch all the possible exception you can `catch` **Throwable**.
- If you want to catch exceptions of both subclass type and superclass type, put the subclass first in the catch sequence.

# Example



```
// Use multiple catch statements
class Example2 {
    public static void main(String args[]) {
        // Here, numer is longer than denom
        int numer[] = {4, 8, 16, 32, 64, 128, 256, 512};
        int denom[] = {2, 0, 4, 4, 0, 8};

        for (int i = 0; i < numer.length; i++){
            try{
                System.out.println(numer[i] + " / " +
                                    denom[i] + " is " +
                                    numer[i]/ denom[i]);
            }
            catch (ArrayIndexOutOfBoundsException exc){
                // Catch the exception
                System.out.println ("No matching element
found.");
            }
            catch (Throwable exc){
                // Catch the exception
                System.out.println ("Some Exception
Occurred");
            }
        }
    }
}
```



# Output

```
4 / 2 is 2
```

```
Some exception occurred.
```

```
16 / 4 is 4
```

```
32 / 4 is 8
```

```
Some exception occurred.
```

```
128 / 8 is 16
```

```
No matching element found.
```

```
No matching element found.
```



# Class Throwable

- Throwable provides:
  - A String to store a message about an exception.
  - A method `String getMessage()` to return the message string.
  - A method `printStackTrace()` to display the stack trace.
  - And a few other methods.
- The Subclasses that extend Throwable can add further variables and methods.



## Some Java Built-in exceptions

- ArithmeticException
- ArrayIndexOutOfBoundsException
- ArrayStoreException
- ClassCastException
- IllegalArgumentException
- IllegalStateException
- IllegalThreadStateException
- IndexOutOfBoundsException
- StringIndexOutOfBoundsException



# Throwing an Exception

- A method can directly or indirectly throw an exception without catching it.
  - the exception must be declared by a **throws declaration** for class Exception and subclasses (excluding RuntimeException).
- The general form is

```
throw exceptObj;
```

- Where *exceptObj* must be an object of an exception class derived from Throwable



# Example

```
public static void parseInt(String s, int num) Throws Exception
{
    if (s == null)
    {
        throw new Exception("null");
    }

    if (num < 0) {
        throw new Exception("negative number");
    }
}
```

Declaration of throws!

Throw Exception object



# Example throwing exceptions

```
// Manually throw an exception
class ThrowExample {
    public static void main(String args[]) {
        try{
            System.out.println("Before throw.");
            throw new ArithmeticException();
        }
        catch (ArithmeticException exc){
            // Catch the exception
            System.out.println ("Exception caught.");
        }
        System.out.println("After try/catch block.");
    }
}
```

Runtime exception,  
You don't need to declare

A red arrow pointing from the text "Runtime exception, You don't need to declare" to the line "throw new ArithmeticException();" in the code.

Output:

```
Before throw.
Exception caught.
After try/catch block.
```

# Creating Exception Subclasses

- You can create your own exceptions.
- Just define a subclass of **Exception**.
- The **Exception** class does not define any methods of its own, it inherits those methods defined by **Throwable**.



# Writing an Exception Class

```
class MyException extends Exception {
```

```
    public MyException ()  
    {  
        super("Default message") ;  
    }
```

```
    public MyException (String s)  
    {  
        super (s) ;  
    }  
}
```

The *extends* keyword specifies that MyException is a *subclass* of Except



## Example – Create an exception

```
// Create an Exception
class NonIntResultException extends Exception{
    int n;
    int d;

    NonIntResultException (int i, int j) {
        n = i;
        d = j;
    }

    public String toString(){
        return "Result of " + n + " / " + d + " is non-
integer";
    }
}
```



# Example using a Custom Exception

```
class CustomExceptionDemo{
    public static void main(String args[]) {

        // Here, numer is longer than denom
        int numer[] = {4, 8, 15, 32, 64, 128, 256, 512};
        int denom[] = {2, 0, 4, 4, 0, 8};

        for (int i = 0; i < numer.length; i++){
            try {
                if(numer[i]%den[i] != 0)
                    throw new NonIntResultException(numer[i], demom[i]);

                System.out.println(numer[i] + " / " +
                                    denom[i] + " is " +
                                    numer[i]/ denom[i]);
            }
            catch (ArithmeticException exc){
                // Catch the exception
                System.out.println ("Can't divide by zero!");
            }
            catch (NonIntResultException exc){
                // Catch the exception
                System.out.println(exc) ;
            }

            catch (ArrayIndexOutOfBoundsException exc){
                // Catch the exception
                System.out.println ("No matching element found.");
            }
        }
    }
}
```



# Output

```
4 / 2 is 2
```

```
Can't divide by zero!
```

```
Result of 15 / 4 is non-integer.
```

```
32 / 4 is 8
```

```
Can't divide by zero!
```

```
128 / 8 is 16
```

```
No matching element found.
```

```
No matching element found.
```